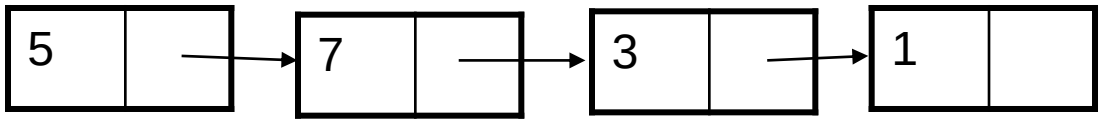
Linked List

- A linked list is a linear collection of data elements (called nodes), where the linear order is given by means of pointers.
- Each node is divided into 2 parts:
  - 1st part contains the information of the element.
  - 2nd part is called the link field or next pointer field which contains the address of the next node in the list.

```
struct node{
    int value;
    struct node *next;
}
```

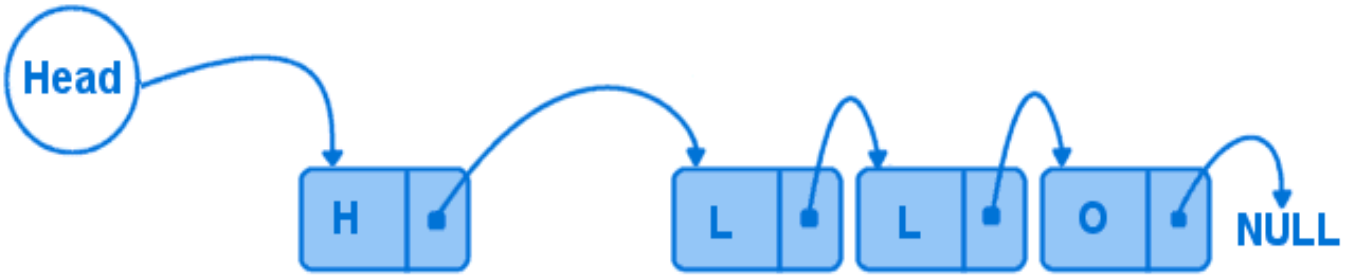


Basic operations

- **Insert**: Add a new node in the first, last or interior of the list.
- **Delete**: Delete a node from the first, last or interior of the list.
- **Search**: Search a node containing particular value in the linked list.

Insertion to a Linear Linked list

- Add a new node at the first, last or interior of a linked list.

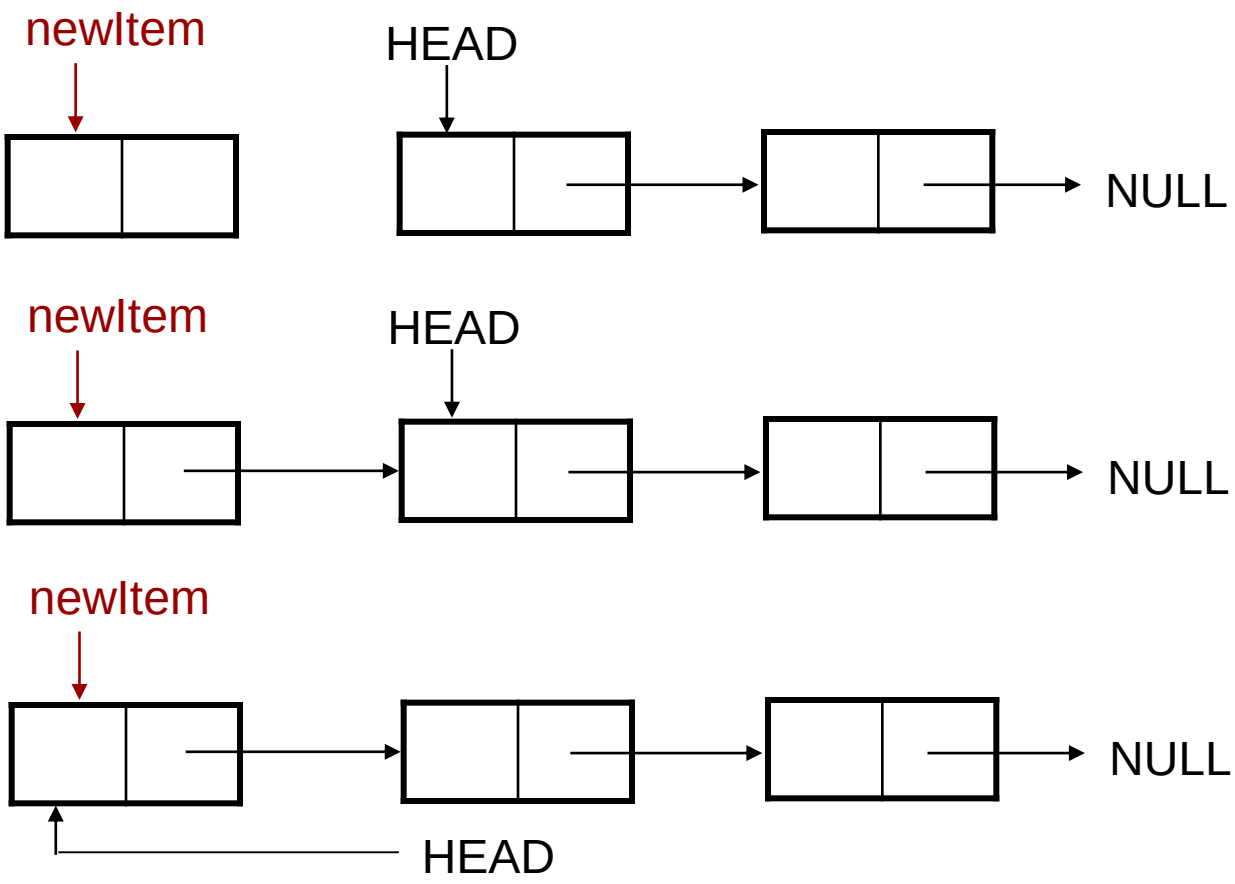


Insert First

- To add a new node to the head of the linear linked list, we need to construct a new node that is pointed by pointer *newItem*.
- Assume there is a global variable *head* which points to the first node in the list.
- The new node points to the first node in the list. The *head* is then set to point to the new node.

Insert First (Cont.)

- Step 1. Create a new node that is pointed by pointer *newItem*.
- Step2. Link the new node to the first node of the linked list.
- Sep3. Set the pointer *head* to the new node.



Insert First (Cont.)

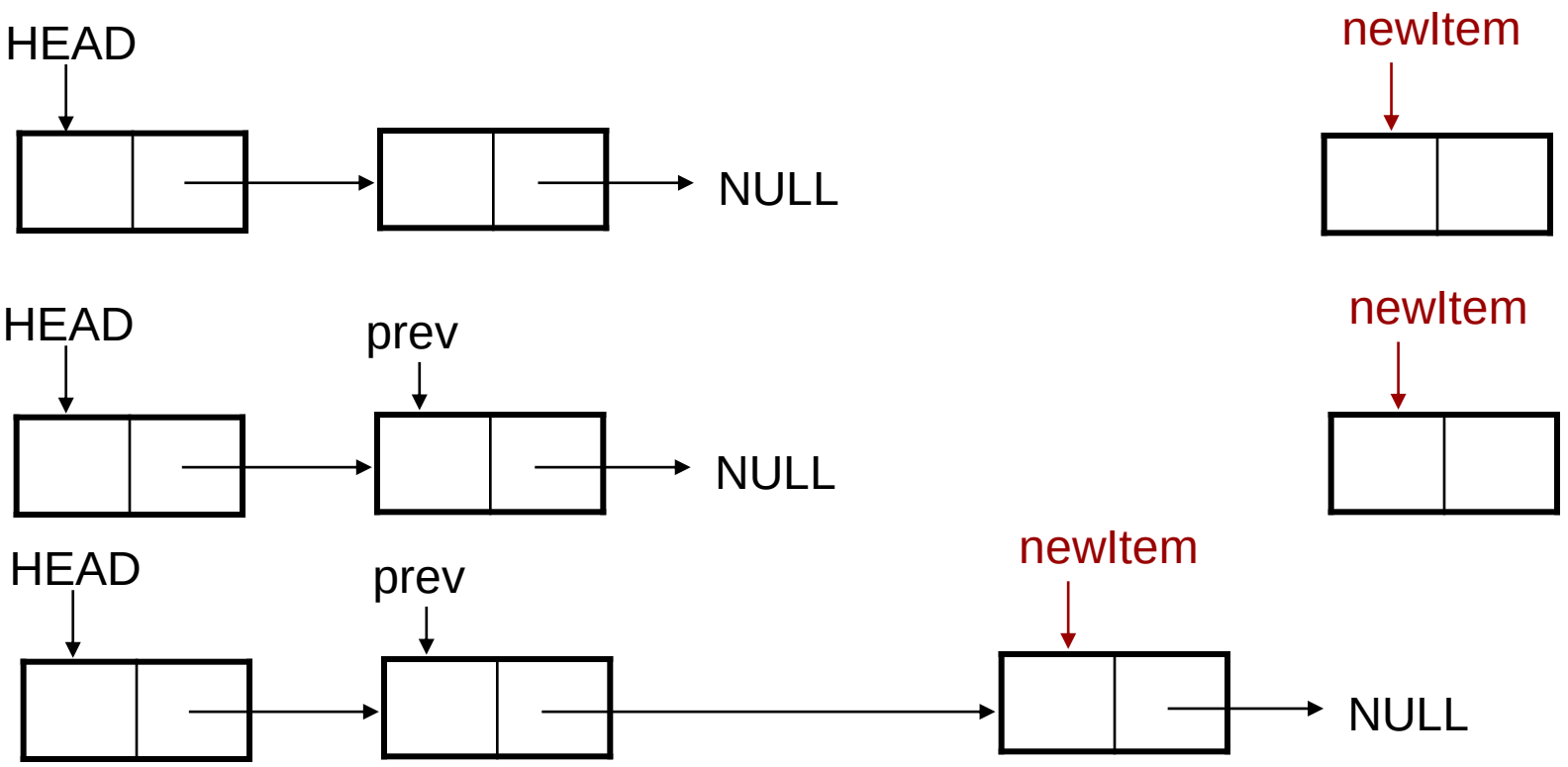
```
struct node{
  int value;
  struct node *next;
};
node *head;

void insertHead(){
   //create a new node
   node *newItem = new node();
   newItem->value = 10;
   newItem->next = NULL;
   //insert the new node at the head
   newItem->next = head;
   head = newItem;
}
```

Insert Last

- To add a new node to the tail of the linear linked list, we need to construct a new node and set it's link field to "null".
- Assume the list is not empty, locate the last node and change it's link field to point to the new node

Insert Last (Cont.)

- Step1. Create the new node.
- Step2. Set a temporary pointer prev to point to the last node.
- Sep3. Set prev to point to the new node and new node as last node.



Insert Last (Cont.)

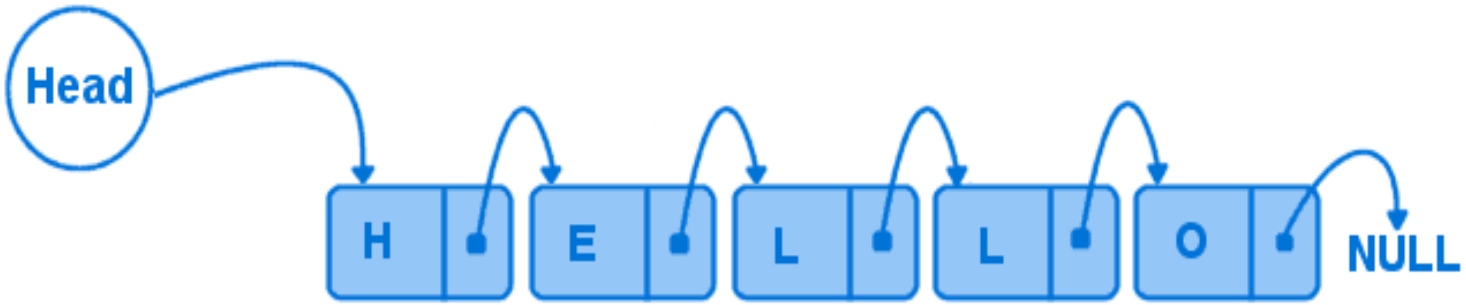
```
struct node{
  int value;
  struct node *next;
};
node *head;

void insertTail(){
   //create a new node to be inserted
   node *newItem = new node();
   newItem->value = 10;
   newItem->next = NULL;
   // set prev to point to the last node of the list
   node *prev = head;
   while (prev->next != NULL){
      prev = prev->next;
   }
   newItem->next = prev->next;
   prev->next = newItem;
}
```

Printing List

```
void printList()
{
  if (head == NULL)   // no list at all
                return;
  node *cur= head;
  while (cur != NULL)
  {
      printf("%d ",cur->value);
              cur=cur->next;
  }
}
```

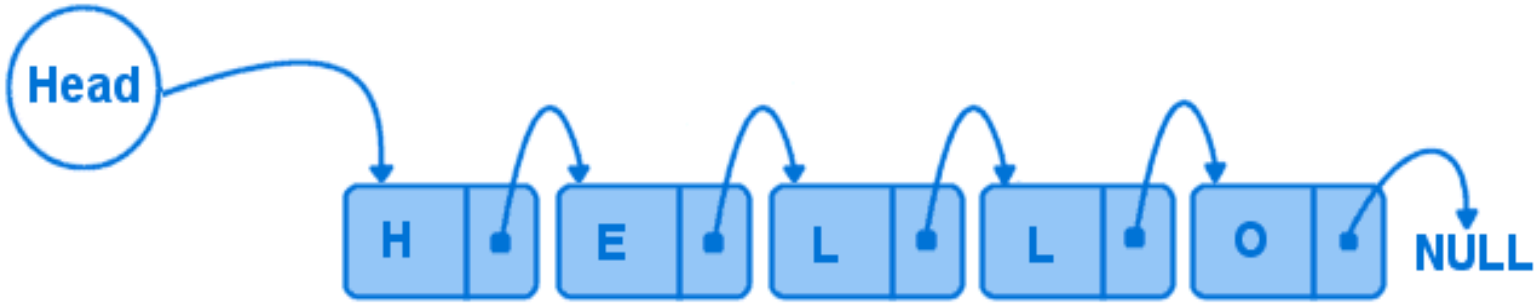
Deletion from a Linear Linked list

- Deletion can be:
  - At the first node of linked list.
  - At the end of a linked list.
  - Within the linked list.



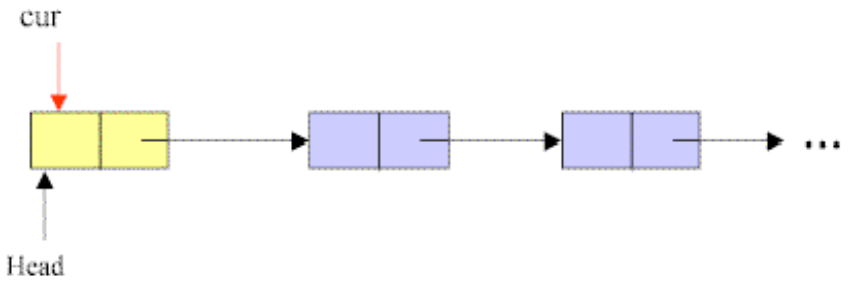
Delete First

- To delete the first node of the linked list, we not only want to advance the pointer *head* to the second node, but we also want to release the memory occupied by the abandoned node.

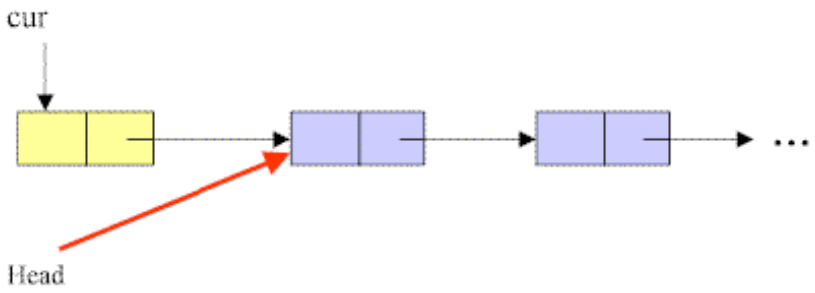


Delete First (Cont.)

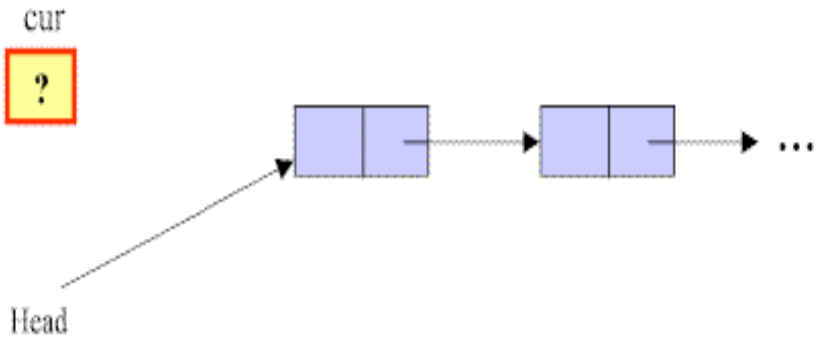
- Step1. Initialize the pointer *cur* point to the first node of the list.
- Step2. Move the pointer *head* to the second node of the list.
- Sep3. Remove the node that is pointed by the pointer *cur*.



Step 1



Step 2



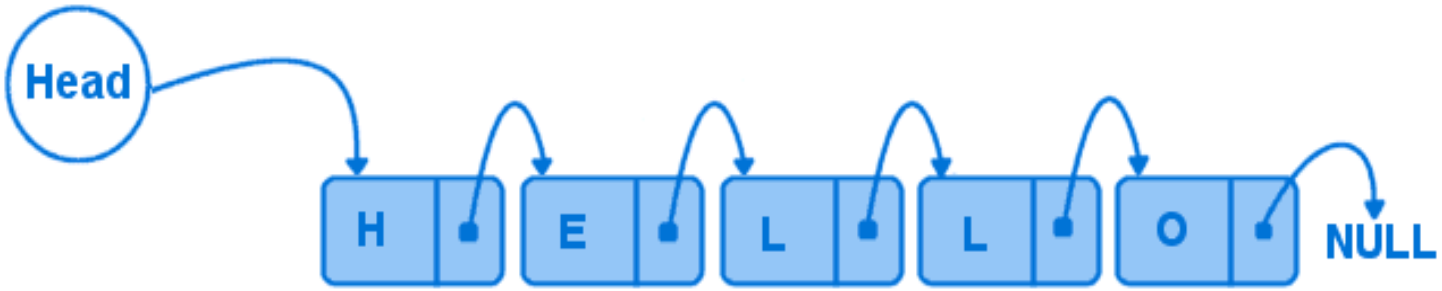
Step 3

Delete First (Cont.)

```
void deleteHead()
{
  if (head == NULL)  //list empty
    return;
  node *cur = head;  // save head pointer
  head = head->next; //advance head
  delete cur;
}
```

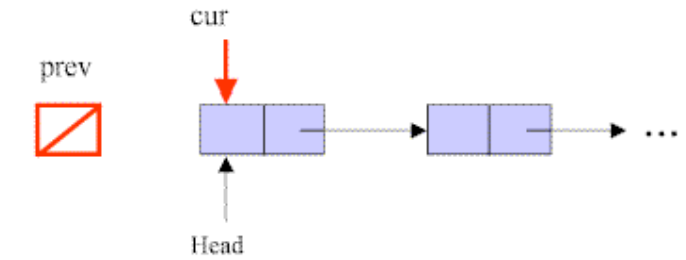
Delete Last

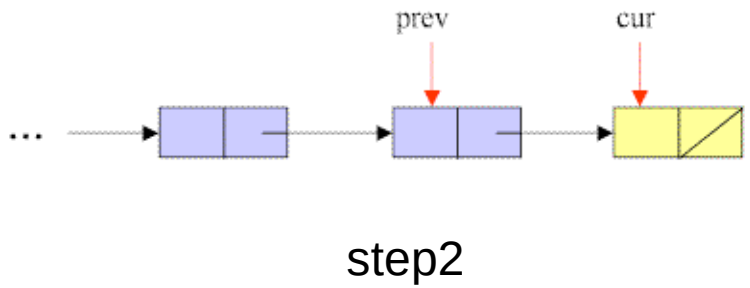
To **delete** the last node in a linked list, we use a local variable, *cur*, to point to the last node. We also use another variable, *prev*, to point to the second last node in the linked list.

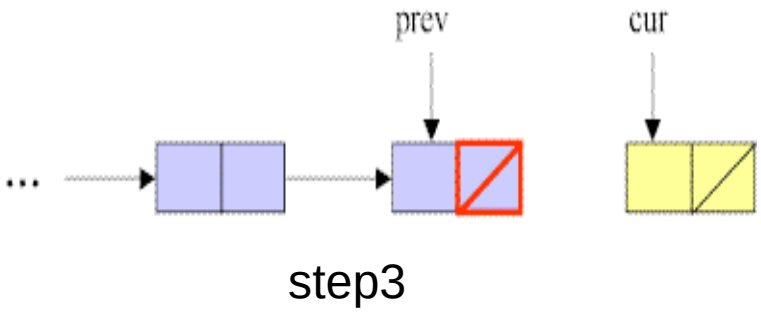


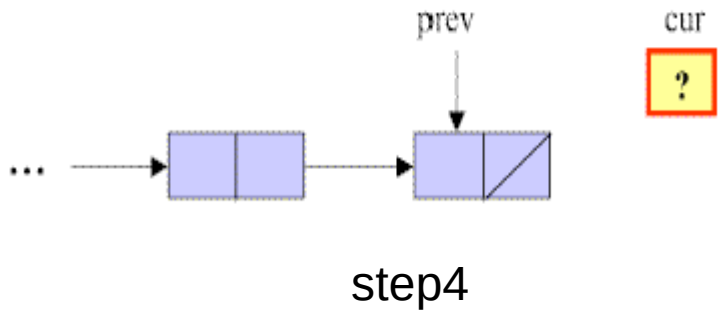
Delete Last (Cont.)

- Step1. Initialize pointer *cur* to point to the first node of the list, while the pointer *prev* has a value of null.
- Step2. Traverse the entire list until the pointer *cur* points to the last node of the list.
- Sep3. Set NULL to *next* field of the node pointed by the pointer *prev*.
- Step4. Remove the last node that is pointed by the pointer *cur*.







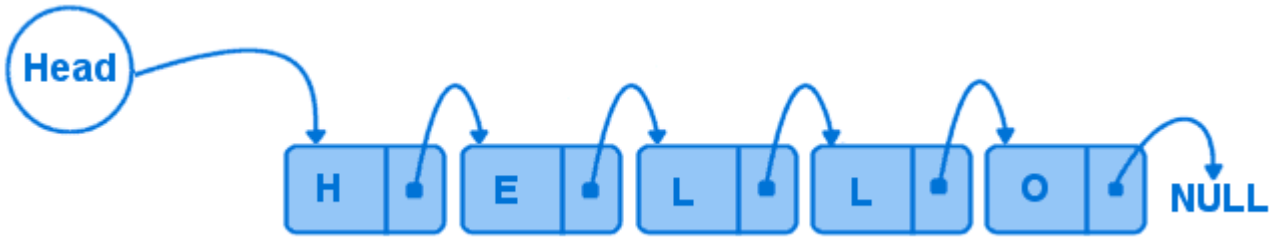


Delete Last (Cont.)

```
void deleteTail()
{
  if (head == NULL)      //list empty
    return;
  node *cur = head;
  node *prev = NULL;
  while (cur->next != NULL)
  {
    prev = cur;
    cur=cur->next;
  }
  if (prev != NULL)
        prev->next = cur->next;       //NULL;
  delete cur;
}
```

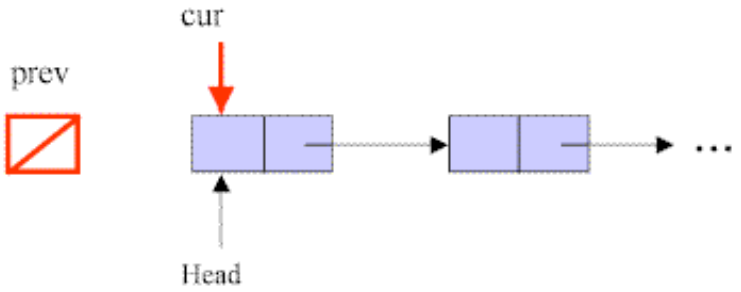
Delete Any

To delete a node that contains a particular value *X* in a linked list, we use a local variable, *cur*, to point to this node, and another variable, *prev*, to hold the previous node.

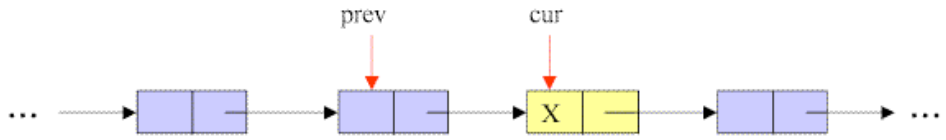


Delete Any (Cont.)

- Step1. Initialize pointer *cur* to point to the first node of the list, while the pointer *prev* has a value of null.
- Step2. Traverse the entire list until the pointer *cur* points to the node that contains value of X, and *prev* points to the previous node.
- ……..



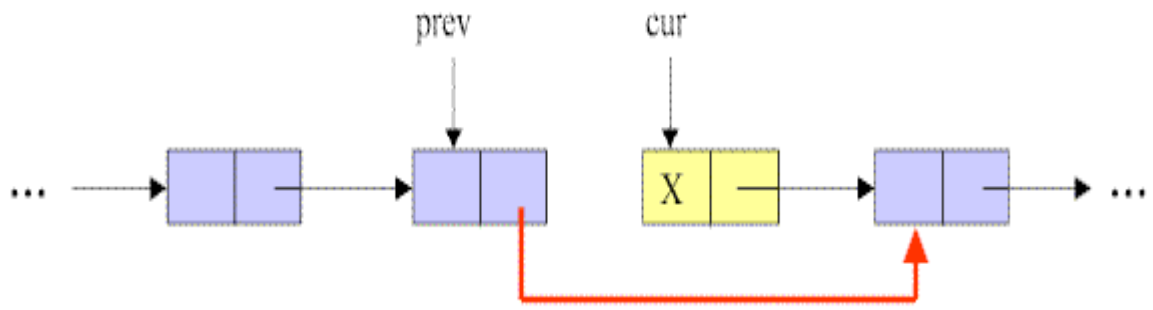
Step 1



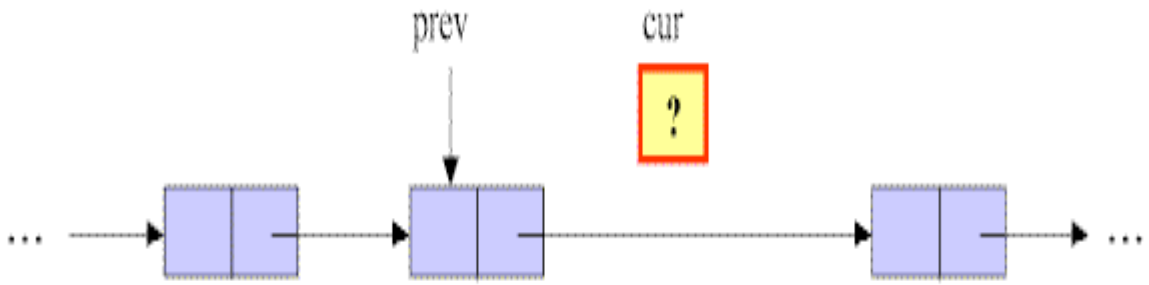
Step 2

Delete Any (Cont.)

- …….
- Step3. Link the node pointed by pointer *prev* to the node after the *cur*'s node.
- Step4. Remove the node pointed by *cur*.



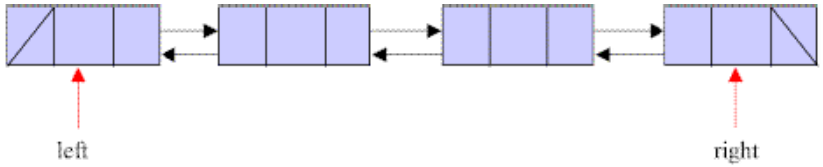
Step 3



Step 4

Introduction to Doubly Linked List

- We have discussed the details of linear linked list. In the linear linked list, we can only traverse the linked list in one direction.
- But sometimes, it is very desirable to traverse a linked list in either a forward or reverse manner.
- This property of a linked list implies that each node must contain two link fields instead of one. The links are used to denote the predecessor and successor of a node. The link denoting the predecessor of a node is called the left link, and that denoting its successor its right link



Basic Operation of Doubly Linked List

- **Insert**: Add a new node in the first, last or interior of the list.
- **Delete**: Delete a node from the first, last or interior of the list.
- **Search**: Search a node containing particular value in the linked list.

Insert First or Insert Last into a Doubly Linked List

- **Insertion** is to add a new node into a linked list. It can take place anywhere -- the first, last, or interior of the linked list.

- To add a new node to the head and tail of a double linked list is similar to the linear linked list.
- First, we need to construct a new node that is pointed by pointer *newItem*.
- Then the newItem is linked to the left-most node (or right-most node) in the list. Finally, the *Left* (or *Right*) is set to point to the new node.

Insert Interior of Doubly Linked List

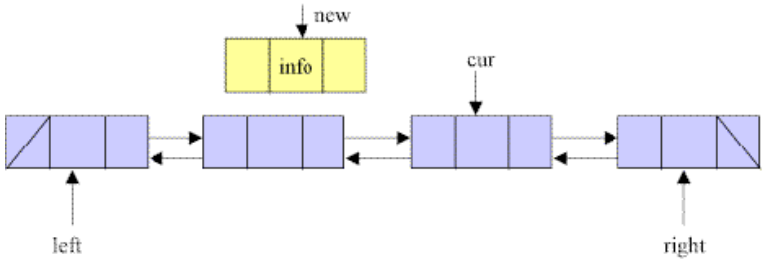
Step 1. Create a new node that is pointed by *newItem*

Step 2. Set the pointer *prev* to point to the left node of the node pointed by *cur*.

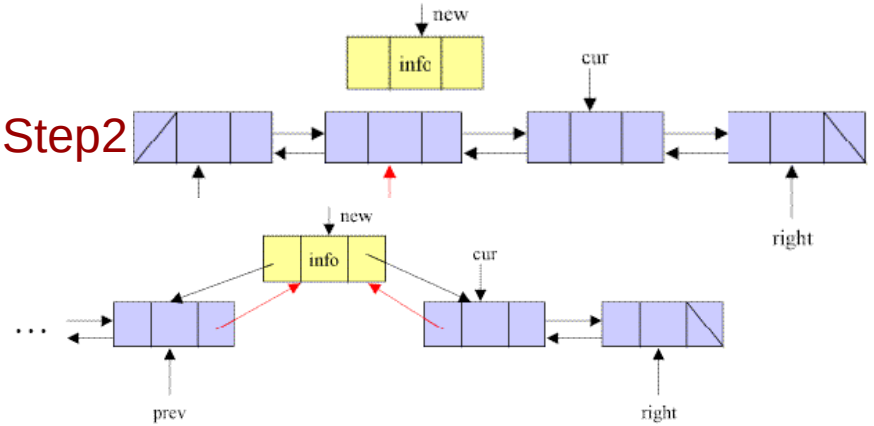
Step3. Set the left link of the new node to point to the node pointed by *prev*, and the right link of the new node to point to the node pointed by *cur*.

Step4. Set the right link of the node pointed by *prev* and the left link of the node pointed by *cur* to point to the new node.

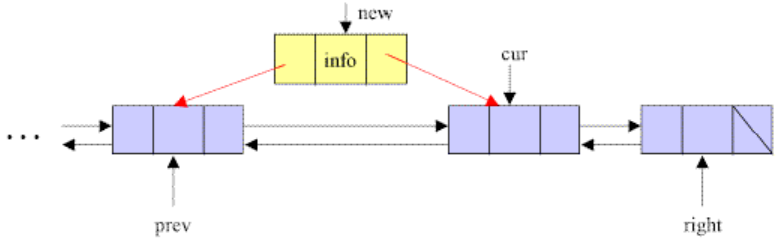
Step1



Step2



Step3



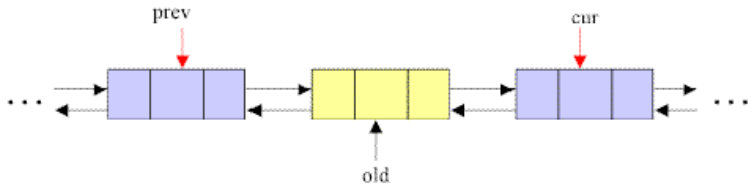
Step4

Deletion of Doubly Linked List

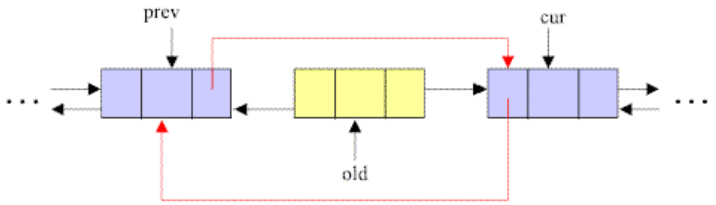
- **Deletion** is to remove a node from a list. It can also take place anywhere -- the first, last, or interior of a linked list.
- To delete a node from a double linked list is easier than to delete a node from a linear linked list.
- For deletion of a node in a single linked list, we have to search and find the predecessor of the discarded node. But in the double linked list, no such search is required.
- Given the address of the node that is to be deleted, the predecessor and successor nodes are immediately known.

Deletion of Doubly Linked List (Cont.)

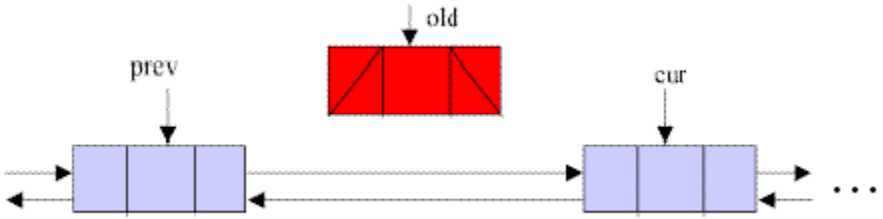
- Step 1. Set pointer *prev* to point to the left node of *old* and pointer *cur* to point to the node on the right of *old*.
- Step2. Set the right link of *prev* to *cur*, and the left link of *cur* to *prev*.
- Step3. Discard the node pointed by *old*.



step1



step2



step3